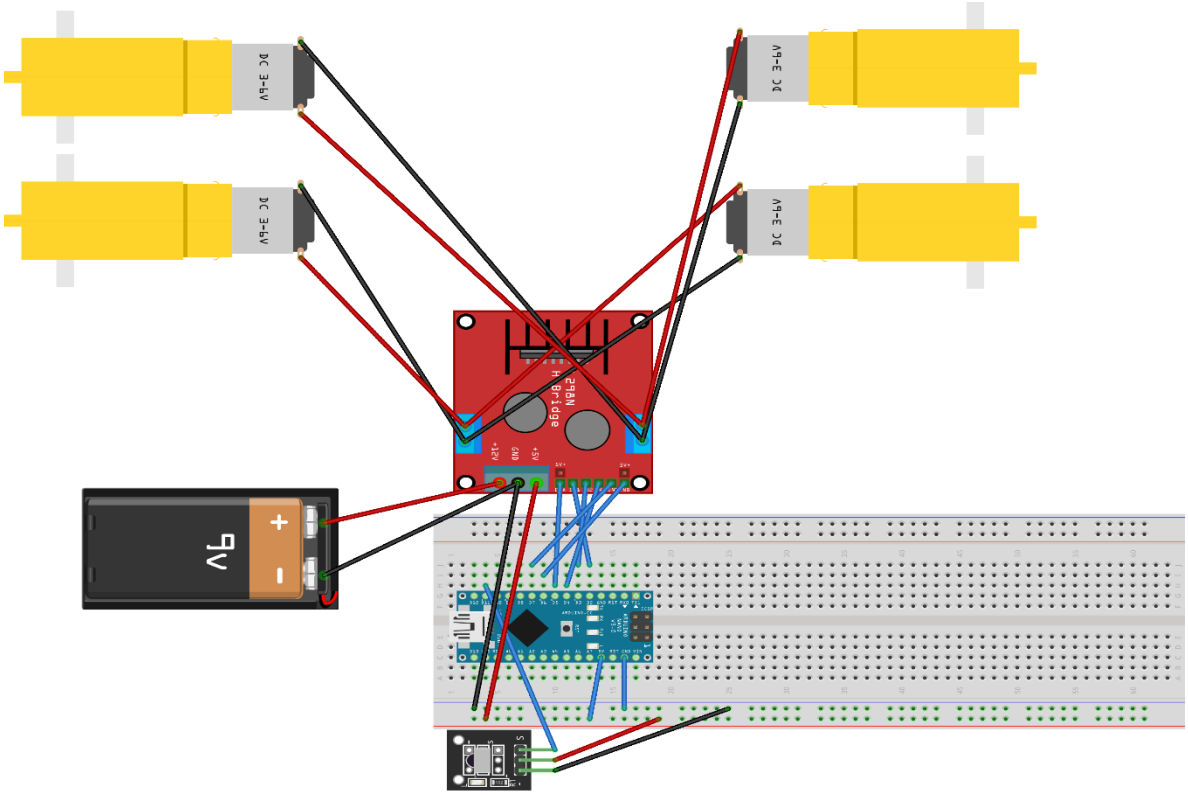
Proyecto IoT

Coche Arduino controlado por sensor de luz infrarroja

Ingrid Evangelista Orta

Tecnología de Computadores IC2

1. Diagrama de conexiones



2. Componentes

- Pila 9v: proporciona energía a todo el sistema.
- Motores DC: generan el movimiento de las ruedas.
- Driver del motor: hace de intermediario entre el Arduino Nano y los motores y permite controlar la dirección del coche.
- Arduino Nano: se encarga de procesar las señales y controlar el resto de componentes.
- Receptor IR: recibe las señales de luz infrarroja emitidas por el mando y las convierte en datos que puede interpretar el Arduino.

3. Código

```
#include <IRremote.h>

// Pines del sensor IR
#define RECV_PIN 11

// Pines del controlador L298N
#define IN1 2
#define IN2 3
#define IN3 4
#define IN4 8
#define ENA 5
#define ENB 7

IRrecv irrecv(RECV_PIN);
decode_results results;

void setup() {
  // Configuración de pines como salida
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);

  // Habilitar motores a velocidad máxima
  analogWrite(ENA, 255); // Motor 1
  analogWrite(ENB, 255); // Motor 2


 // Inicializar receptor IR
  irrecv.enableIRIn();
```

```
  // Inicializar comunicación serie para depuración
  Serial.begin(9600);
}

void loop() {
  // Verificar si se ha recibido un código IR
  if (irrecv.decode(&results)) {
    unsigned long command = results.value;
    Serial.println(command, HEX); // Mostrar el comando recibido
en formato hexadecimal

    // Controlar los motores según el comando IR
    controlMotors(command);

    irrecv.resume(); // Preparar para recibir el próximo comando
  }
}

void controlMotors(unsigned long command) {
  switch (command) {
    case 0xFFA25D: // Adelante
      moveForward();
      break;
    case 0xFF629D: // Atrás
      moveBackward();
      break;
    case 0xFFE21D: // Izquierda
      turnLeft();
      break;
```

```
case 0xFF22DD: // Derecha
      turnRight();
      break;
    case 0xFF02FD: // Detener
      stopMotors();
      break;
    default:
      stopMotors(); // Detener si el comando no es reconocido
      break;
  }
}

void moveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void moveBackward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}
void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}
```

```
void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);}

void stopMotors() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}
```

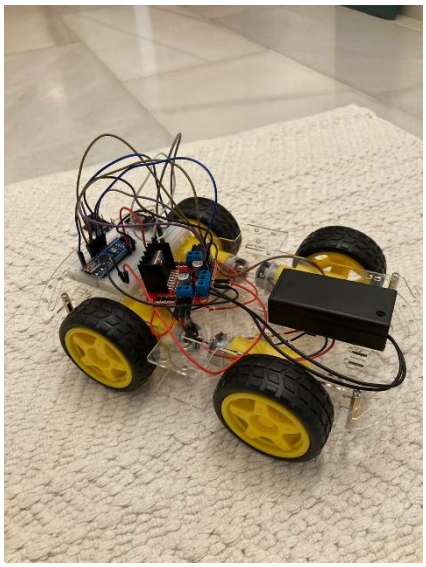
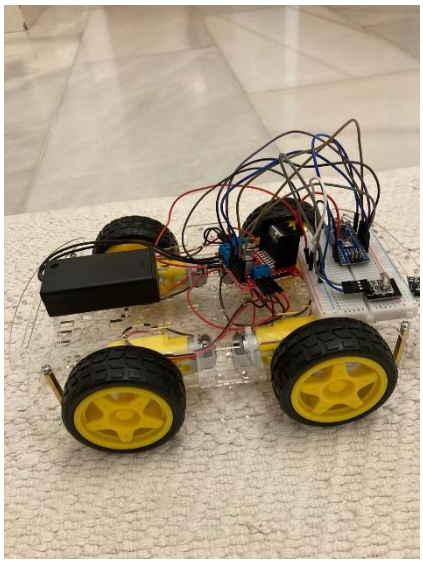
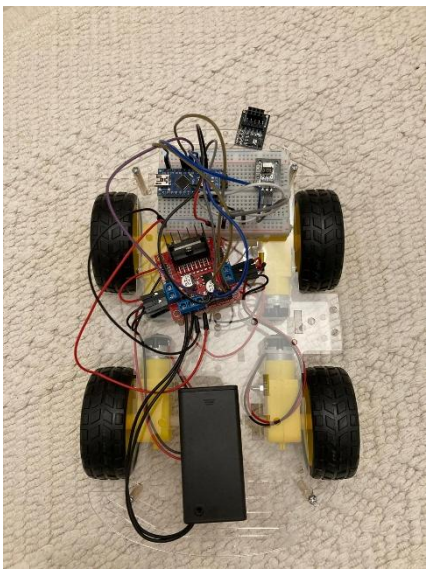
En la sección setup se configuran los pines asociados al controlador de motores L298N como salidas, permitiendo que el Arduino gestione las señales que controlan el movimiento del coche. También se habilita el receptor infrarrojo (IR) conectado al pin 11 para recibir señales del mando y se inicia la comunicación serie, lo que facilita la depuración al mostrar en el monitor serie los comandos recibidos. Además, los motores se configuran inicialmente a velocidad máxima mediante el uso de la función analogWrite.

En el bucle principal (loop), el Arduino verifica constantemente si el receptor IR ha recibido una señal del mando a distancia. Si se detecta un comando, este se almacena en la variable command y se imprime en formato hexadecimal en el monitor serie para fines de depuración. Luego, el comando recibido se pasa a la función controlMotors, que se encarga de interpretar el código y ejecutar la acción correspondiente.

La función controlMotors utiliza una estructura switch para comparar el comando recibido con una lista de códigos predefinidos. Según el código, se llama a la función que controla el movimiento específico del coche:

- moveForward(): hace que el coche avance activando los pines correspondientes para que ambos motores giren hacia adelante.
- moveBackward(): permite que el coche retroceda invirtiendo el sentido de giro de ambos motores.
- turnLeft(): hace que el coche gire a la izquierda activando el motor derecho y deteniendo el izquierdo.
- turnRight(): gira el coche hacia la derecha activando el motor izquierdo y deteniendo el derecho.
- stopMotors(): detiene completamente el coche apagando ambos motores.

Si el comando recibido no coincide con ninguno de los códigos predefinidos, la función stopMotors se ejecuta como medida de seguridad, deteniendo el coche para evitar comportamientos imprevistos.

4. Controles

5. Imágenes